Bangladesh Army University of Engineering & Technology (BAUET)
Qadirabad Cantonment, Natore-6431



Department of Computer Science and Engineering (CSE)

Project Report

Course Code	CSE-4205
Course Title	Computer Graphics Sessional
Project Title	Moto Racer 3D: Highway Race Simulator

Date of Submission	26-10-2026

Submitted by		**Submitted to**	
ID	0812220205101120	Name:	Md. Nazmus Salehin
Name	Md. Mehedi Hasan Ridoy	Designation	Lecturer
Batch	16th	Dept.	CSE, BAUET
Year	4th		
Semester	2nd	Name:	Mst. Irin Sultana
Section	B	Designation	Lecturer
Session	2022-23	Dept.	CSE, BAUET

Project Name: Moto Racer 3D: Highway Race Simulator

1.1 Objectives

The main objectives of the project are as follows:

i. To design and develop a standalone 3D highway motorbike racing game using C++, OpenGL, and GLUT without relying on external 3D models, image textures, or audio files.

ii. To create a low-poly city highway containing buildings, shop awnings, trees, street lamps, sidewalks, lane markings, and a finish arch using only basic geometric primitives such as GL_QUADS and GL_POLYGON.

iii. To implement a Highway-Rider style third-person chase camera placed behind and above the player's bike, using a perspective projection (glFrustum), so that the bike is always clearly visible in the lower part of the screen.

iv. To generate a procedurally curved road from a mathematical function so that the track bends left and right without storing any map data.

v. To implement responsive, frame-rate-independent controls for acceleration, braking, steering, nitro boost, and side kicks by using the real elapsed time of each frame.

vi. To simulate 50 traffic vehicles (cars and trucks) moving along three lanes and to detect collisions between bikes and traffic using interval-based and lateral-distance checks.

vii. To develop three computer-controlled riders that avoid traffic, adjust their speed according to the player's position, use nitro, and fight with rivals.

viii. To implement a health, damage, stun, and respawn system together with a nitro energy system.

ix. To design a 2D Heads-Up Display (HUD) using orthographic projection that shows speed, nitro, health, race ranking, race progress, start lights, and the final result medal.

x. To implement a race state machine (countdown, racing, finished) with ranking calculation, restart, and automatic restart after the race.

1.2 Introduction

Moto Racer 3D: Highway Race Simulator is an interactive 3D motorbike racing game developed using C++, OpenGL, and GLUT. The player rides a sports bike on a long city highway against three computer-controlled riders while avoiding slow-moving cars and trucks. The race is 2200 units long and the winner is the rider who first crosses the checkered finish line under the finish arch.

The entire game world is created from basic OpenGL primitives. Buildings, trees, street lamps, vehicles, the motorbikes, and even the riders are assembled from coloured boxes, while round

objects such as the sun, nitro flames, and start lights are drawn as polygons. The game does not use any external 3D model, texture image, or audio file. Face shading is applied manually by darkening the side faces of each box, which gives a solid 3D appearance without using the OpenGL lighting system.

The gameplay combines racing with light combat. Players accelerate, brake, and steer between lanes, collect speed using a limited nitro boost, and may kick rival riders who are beside them to push them away and reduce their health. Collisions with traffic reduce the rider's health and speed. If health reaches zero, the rider crashes, remains stunned for a short time, and then respawns with partial health. The current position, speed, health, nitro level, and race time are shown through a HUD and the window title.

The primary purpose of this project is to apply the theoretical knowledge of computer graphics to develop a complete real-time game. It covers geometric modelling, hierarchical transformations, perspective and orthographic projection, depth testing, procedural generation, animation, collision detection, simple artificial intelligence, and game state management.

1.2.1 Design and Implementation

The game is designed and implemented using the following techniques:

- **Programming Environment:** Developed in C++ using OpenGL and GLUT for rendering, window management, keyboard input, and timer-based animation. Only fixed-function OpenGL calls such as glBegin, glEnd, glVertex3f, glColor3f, glPushMatrix, glPopMatrix, glTranslatef, glRotatef, and glScalef are used for drawing.
- **Geometric Modelling:** A reusable box() function draws a cuboid with five coloured faces whose brightness is scaled differently for the top, rear, front, right, and left sides. The part() function places a box at a given position using glTranslatef. Every building, vehicle, tree, lamp, and motorbike is built by combining these functions.
- **Hierarchical Animation:** The rider's legs use nested glPushMatrix() and glPopMatrix() transformations so that a leg can rotate about the hip during a kick. The whole bike is rotated about its z-axis to lean during turns, and rotated by 80 degrees when the rider is crashed and lying on the road.
- **Procedural Road and City:** The road centre line is calculated by a mathematical function of the distance travelled. Buildings, trees, and lamps are placed every 14 units on both sides of the road, and their size and colour are selected by an integer hash function, so the city can be generated again identically without storing any data.
- **Camera System:** A perspective view is created using glFrustum. The camera is placed 9 units behind and 4.4 units above the bike, looks slightly downward, turns with the direction of the road, and follows the player's lateral position with a smoothed delay.

- **Rendering Order:** The sky gradient and the flat ground (grass, sidewalks, asphalt, lane lines) are drawn first with depth testing disabled. Depth testing is then enabled for all real 3D objects. The HUD is drawn last using an orthographic projection.
- **Real-Time Update Loop:** A GLUT timer callback runs every 8 ms. The real elapsed time is read using glutGet(GLUT_ELAPSED_TIME), limited to 0.05 s, and passed to the update function so that the game speed does not depend on the computer.
- **Traffic and Collision Detection:** Fifty vehicles are generated in three lanes. Collisions between bikes and vehicles are checked using the distance along the road and the lateral distance. Bike-to-bike collisions use a small separation push.
- **Artificial Intelligence:** Each AI rider looks ahead for traffic, steers around it, adjusts its speed, occasionally uses nitro, and attacks rivals who are beside it.
- **Game State Management:** A state variable controls the countdown, the race, and the finished screen. Keyboard events are handled with both key-down and key-up callbacks so that held keys give smooth control.

Table 1.1 summarises the main entities of the game and their important parameters.

Table 1.1: Main Game Entities and Their Parameters in Moto Racer 3D

Entity	**Count**	**Main Properties**
Player bike	1	Controlled with W, A, S, D; wanted speed up to 32 units/s (displayed speed = 4 × speed in km/h); lateral speed 16 units/s; can leave the road up to ±8.5 units; speed is limited to 12 units/s when |u| > 6 (off road).
AI bikes	3	Base wanted speed 22 + i units/s with a catch-up bonus between −2 and +4; lateral speed 9 units/s; lateral limit ±5; start with 100 health and 100 nitro.
Cars	About 5 of every 6 vehicles	Length 4.4, width 2.0, height 1.5; speed 10 to 15 units/s; random body colour.
Trucks	About 1 of every 6 vehicles	Length 8.0, width 2.5, height 3.0; speed 8 to 10 units/s; grey container and coloured cabin.
Traffic lanes	3	Lane positions −4, 0, and +4 from the road centre; total traffic vehicles NCARS = 50.
Track	1	Length 2200 units; road width 12 units; sidewalks from 6 to 9 units; first 300 units are straight; start line at 0 and finish line at 2200.

Entity	Count	Main Properties
Buildings, trees, lamps	One set every 14 units, both sides	Building width 7 to 12, height 5 to 13, depth 9; windows, door, awning, and roof sign; trees and lamps selected by a hash function.

1.2.2 Road Generation and Environment Modelling

The road is not stored as a map. Instead, the horizontal position of the road centre at any distance s is calculated by the function roadX(s). The road is straight for the first 300 units and then gradually bends by blending two sine waves:

$$\text{roadX}(s) = t^2 \cdot (13 \cdot \sin(0.012\ s) + 5 \cdot \sin(0.03\ s)), \quad \text{where } t = \text{clamp}(s\ /\ 300,\ 0,\ 1)$$

The factor t^2 makes the bending start smoothly, so that the race begins on a straight road. Every road element is drawn as short 4-unit strips. For each strip the left and right edges are calculated using the road centre at the beginning and at the end of the strip, which creates continuous curves. The strips are drawn in the following order: grass (±80 units), sidewalks, asphalt, white side lines, and dashed lane lines at ±2 units.

The direction of the road at distance s is obtained from the slope of the centre line:

$$\theta(s) = \arctan[\ (\text{roadX}(s + 3) - \text{roadX}(s - 3))\ /\ 6\ ]$$

This angle is used to rotate cars and bikes so they follow the bends of the road. Only the range of the world near the player (from 20 units behind to 260 units ahead for the road and 200 units ahead for objects) is drawn in every frame, which keeps the game fast.

1.2.3 Camera System

The camera is designed in the style of the Highway Rider arcade game. It is controlled by three constants that can be adjusted easily: CAM_DIST = 9.0 (distance behind the bike), CAM_HEIGHT = 4.4 (height above the road), and CAM_PITCH = 6° (downward angle). The projection is created with glFrustum(−0.888, 0.888, −0.577, 0.577, 1, 350), which gives a vertical field of view of about 60° and an aspect ratio matching the 1000 × 650 window.

To make the bike move visibly on the screen when the player steers left or right, the camera does not stick rigidly to the bike. Its lateral position follows the bike with an exponential delay, which does not depend on the frame rate:

$$\text{camU} \leftarrow \text{camU} + (0.7 \cdot u - \text{camU}) \cdot (1 - e\hat{}(-8\ \Delta t))$$

The camera also rotates with the direction of the road. The road direction angle and the camera position are calculated as follows:

$$\theta = \arctan[\ (\text{roadX}(s + 6) - \text{roadX}(s - 6))\ /\ 12\ ]$$

$$\text{camX} = \text{roadX}(\text{s}) + \text{camU} - \text{D} \cdot \sin\,\theta\ , \quad \text{camZ} = -(\text{s} - \text{D} \cdot \cos\,\theta)$$

The view matrix is created with glRotatef(CAM_PITCH, 1, 0, 0), glRotatef(θ in degrees, 0, 1, 0), and glTranslatef. Because of this camera position the player's bike appears large in the lower part of the screen, and the road, traffic, and buildings ahead remain visible.

1.2.4 Bike Movement, Traffic, and Collision Detection

Each bike stores its distance along the road (s), its lateral position (u), current speed, wanted speed, health, nitro, and other timers. The forward and lateral movements are handled separately.

- **Forward Movement:** The speed moves towards the wanted speed with an acceleration of 14 units/s² and a deceleration of 18 units/s². Holding W increases the player's wanted speed and holding S decreases it, both at 16 units/s², up to a maximum of 32 units/s. Nitro adds 12 units/s to the wanted speed for 1.5 seconds.
- **Lateral Movement:** When A or D is held, the target lateral position is set 4 units ahead of the current position, so the bike starts moving immediately at 16 units/s. When the key is released, the target is set to the current position and the bike stops at once. This removes the slow, delayed steering of the earlier version.
- **Leaning:** The lean angle is calculated from the lateral velocity, limited to ±22 degrees, and smoothed, so the bike leans realistically while changing lanes.
- **Bike and Traffic Collision:** A collision with a vehicle occurs when the bike is within the vehicle's length along the road and within its width across the road:

$$|\text{s_car} - \text{s_bike}| < \text{L}/2 + 1 \quad \text{and} \quad |\text{u_car} - \text{u_bike}| < \text{W}/2 + 0.6$$

If the bike is behind the vehicle, the bike is placed behind it, its speed is limited to half of the vehicle's speed, and 15 damage is applied. If the bike is beside the vehicle, it is pushed sideways. The following code shows the main collision logic:

```
if (fabsf(rel) < half && fabsf(car[k].u - b.u) < car[k].wid / 2 + 0.6f)
{
    if (rel > 0) {                      // hit the back of a car
        b.s = car[k].s - half;
        b.speed = fminf(b.speed, car[k].speed * 0.5f);
        hurt(b, 15);
    } else {                            // beside the car
        b.u += (b.u > car[k].u ? 1 : -1) * 6 * dt;
    }
}
```

Here, rel is the distance of the car in front of the bike, and half is half of the car length plus one unit. Bike-to-bike collisions are detected when two bikes are closer than 2.2 units along the road and 1.0 unit across the road; in this case both bikes are pushed slightly apart.

1.2.5 Rider AI, Health, and Combat System

The three AI riders are updated in the same way as the player, but their wanted speed and wanted lateral position are decided by the ai() function:

- **Speed Control:** The wanted speed of AI rider i is 22 + i units/s plus a bonus of −2 to +4 units/s according to the distance between the AI rider and the player. A rider far behind the player catches up, while a rider far ahead slows slightly. This keeps the race close.
- **Traffic Avoidance:** The AI rider checks all vehicles within 20 units ahead and within 2.6 units laterally. If a vehicle is found, the rider moves 3.5 units to the free side (not leaving the ±5 road limit) and reduces its speed to the vehicle's speed plus 2.
- **Rival Attack:** When there is no danger, the AI occasionally moves beside a nearby rival. If a rival is within 3.2 units along the road and between 0.3 and 3.3 units laterally, the AI kicks with a small random probability.
- **Nitro Use:** An AI rider with more than 80 nitro uses the boost at random times.

The kick animation rotates the selected leg up to 80 degrees using a sine curve over 0.3 seconds. A successful kick pushes the opponent 1.8 units sideways, reduces its speed by 5 units/s, shows a yellow hit star, and deals damage. Table 1.2 shows the health, damage, and boost parameters.

Table 1.2: Health, Damage, Kick, and Nitro Parameters

Event / Parameter	**Effect**
Hit from the back of a vehicle	15 damage; speed limited to 50% of the vehicle speed.
Successful kick	12 damage; 1.8 units lateral push; 5 units/s speed loss; hit star for 0.3 s.
Kick timing	Animation 0.3 s; cooldown 0.6 s; reach 3.2 units along the road and 0.3 to 3.3 units sideways.
Invulnerability	0.5 s after every damage to prevent repeated damage.
Health reaches zero	Bike falls (rotated 80°) and is stunned for 2.5 s; speed becomes 0; health is restored to 60.
Nitro boost	Costs 30 nitro; adds 12 units/s for 1.5 s; nitro regenerates at 4 per second up to 100; flame polygons are drawn at the exhaust.
Off-road driving (player)	When $\|u\| > 6$ the wanted speed is limited to 12 units/s.

1.2.6 Race Flow, Game States, and User Interface

The race is controlled by a state variable. Table 1.3 describes the three states.

Table 1.3: Game States in Moto Racer 3D

State	Name	Description
0	Countdown	Lasts 3.5 s. Three start lights are shown; one red light is added each second, and all three turn green at 3.0 s. Bikes stand on the same start line in four lanes (u = 1.5, −1.5, −4.5, and 4.5).
1	Racing	Controls, AI, traffic, collisions, and timers are active. The race time is counted.
2	Player Finished	Starts when the player passes s = 2200. The finishing place is stored, the player slows down, and a result box with a medal appears. The game restarts after 8 s or when R is pressed.

The ranking is calculated every frame. A rider who has finished is ahead of all riders who have not finished and earlier finishers are ahead of later finishers. Otherwise, the rider with the larger distance s is ahead.

The HUD is drawn with an orthographic projection (glOrtho(0, 1000, 0, 650, −1, 1)) over the 3D scene with depth testing disabled:

- **Status Bars (bottom right):** Three bars display speed (green), nitro (blue), and health (colour changes from green to red).
- **Ranking Board (top left):** Four coloured squares, sorted by race position, each with a health bar.
- **Race Progress Bar (top centre):** A line with four coloured markers shows how far each rider has travelled out of 2200 units.
- **Start Lights:** Three circles in a dark box at the centre of the screen during the countdown.
- **Result Medal:** Gold, silver, bronze, or grey disc, depending on the final position, with white squares equal to the position number and the text "Press R to restart".
- **Window Title:** Shows place, speed in km/h, health, nitro, and elapsed race time.

1.3 Challenges and Solutions

Several technical challenges were addressed during the development of Moto Racer 3D. The main challenges and their solutions are described below.

- Challenge 1: Poor View and Hidden Bike

  The first camera was close to the road and the bike appeared small and was not clearly visible, so the player could not judge traffic ahead.

Solution: The camera was redesigned in Highway-Rider style: it was moved back to 9 units, raised to 4.4 units, and given a 6° downward pitch. The bike was scaled to 1.4 times its original size, so it appears large in the lower part of the screen.

- Challenge 2: Slow Response When Moving Left and Right

The player's bike responded slowly to the A and D keys, which made avoiding traffic difficult.

Solution: The fixed time step of 0.016 s was replaced with the real frame time obtained from glutGet(GLUT_ELAPSED_TIME). On Windows the timer often runs slower than requested, so the fixed step made the whole game run slower. The steering was also changed so that the target lateral position is set directly ahead of the bike while a key is held, and the bike stops immediately when the key is released.

- Challenge 3: Road Bends Pushed the Player Sideways

In the first version the player's lateral position was shifted every frame to compensate for the road bend. As a result the road seemed to slide away from the bike and the player could not keep a lane.

Solution: The player was treated like every other rider: the lateral position u is measured from the road centre, so the bike automatically follows the curve. Bikes and cars are rotated with the road angle θ(s) to match the bends visually.

- Challenge 4: Bike Did Not Move on the Screen

When the camera was locked to the bike, steering left and right did not change the bike's screen position, and the game felt stiff.

Solution: The camera's lateral position follows the bike with an exponential smoothing factor (1 − e^(−8Δt)), so the bike moves naturally on the screen and the camera catches up smoothly.

- Challenge 5: Correct Drawing Order Without Texture or Lighting

Large flat road surfaces and 3D objects could overlap incorrectly, especially at long distances.

Solution: The sky and flat ground were drawn first with depth testing disabled, then depth testing was enabled for buildings, vehicles, and bikes. Parts of the bike nearer to the camera, such as the rear wheel and exhaust, are drawn last.

- Challenge 6: Unfair Start Positions

At the beginning, the opponents were placed ahead of the player at different distances, so the race did not start in a fair line.

Solution: All four bikes now start with the same distance (s = 1) in four separate lanes, and a countdown with start lights prevents anyone from moving before the green signal.

- Challenge 7: Finished Screen Looked Like a Frozen Game

  After the player crossed the finish line, the result box stayed on the screen and the game looked stopped.

  Solution: The result box now shows the text "Press R to restart", and the game also restarts automatically after 8 seconds.

- Challenge 8: Keeping the Race Competitive

  If the AI riders always used the same speed, they would either leave the player behind or never challenge the player.

  Solution: A catch-up term based on the distance to the player was added to the wanted speed of AI riders, together with random nitro use and random attacks, which keeps the race close and unpredictable.

These solutions improved the game's view, controls, movement, and overall gameplay experience.

Source Code:

```
//
============================================================
=============
//  MOTO RACER 3D  (fixed version: Highway-Rider style
camera)
//
//  Compile (Linux): g++ moto_racer_simple.cpp -o moto -
lGL -lGLU -lglut -lm
//  Windows        : g++ moto_racer_simple.cpp -o
moto.exe -lfreeglut -lopengl32 -lglu32
//
//  Keys (HOLD them):  W = speed up   S = brake   A =
left   D = right
//         SPACE = nitro   Z = kick left     X = kick
right
//         R = restart     ESC = quit
//
============================================================
=============
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

#define PI    3.14159265f
#define TRACK 2200.0f      // race length
#define NCARS 50           // traffic cars

const int W = 1000, H = 650;

//
------------------------------------------------------------
-------- camera tuning
const float CAM_DIST   = 9.0f;    // how far behind the
bike (bigger = bike smaller)
const float CAM_HEIGHT = 4.4f;    // camera height
const float CAM_PITCH  = 6.0f;    // look-down angle
(smaller = bike lower on screen)

//
------------------------------------------------------------
-------- data
struct Bike {
    float s, u;                    // s = distance on
road, u = left/right position
    float speed, target;           // current speed,
wanted speed
    float health, nitro, boost;
    float wantU, prevU, lean;
    float kickTimer, cooldown, stun, hit, invul;
    int   kickSide, done, place;
    float r, g, b;
};
struct Car { float s, u, speed, len, wid, hgt, r, g,
b; };

Bike bike[4];
Car  car[NCARS];
int   state = 0;                   // 0 = countdown, 1 =
racing, 2 = player finished
int   finishedCount = 0;
float clockT = 0, raceTime = 0;
int   frameNo = 0;
bool  keys[256];                   // which keys are
held down right now
float frameDt = 0.016f;            // real frame time
float camU = 0;                    // smooth camera side
position
int   lastSteer = 0;
float endT = 0;                    // time since player
finished

float clampf(float v, float lo, float hi) { return v <
lo ? lo : (v > hi ? hi : v); }

//
------------------------------------------------------------
-------- road shape
// x position of the road centre at distance s  (road
bends left and right)
float roadX(float s) {
    float t = s / 300.0f;                  // straight
at the start
    if (t < 0) t = 0;
    if (t > 1) t = 1;
    return t * t * (13.0f * sinf(s * 0.012f) + 5.0f *
sinf(s * 0.03f));
}

// road direction (degrees) at distance s
float roadAngle(float s) {
    return atanf((roadX(s + 3) - roadX(s - 3)) / 6.0f) *
180.0f / PI;
}

//
------------------------------------------------------------
-------- game setup
void resetGame() {
    state = 0; clockT = 0; raceTime = 0; finishedCount =
0;
    lastSteer = 0;
    endT = 0;

    float startU[4] = { 1.5f, -1.5f, -4.5f, 4.5f };   //
four lanes side by side
    float startS[4] = { 1, 1, 1,
1 };                  // all on the same start line
    float col[4][3] = { {0.9f,0.1f,0.1f},
{0.1f,0.4f,0.95f}, {0.95f,0.75f,0.1f},
{0.2f,0.8f,0.3f} };
    for (int i = 0; i < 4; i++) {
        bike[i] =
Bike();                              // all zero
        bike[i].s = startS[i];  bike[i].u = startU[i];
bike[i].wantU = startU[i];
        bike[i].health = 100;   bike[i].nitro = 100;
```

```
        bike[i].r = col[i][0];  bike[i].g = col[i][1];
bike[i].b = col[i][2];
    }
    camU = bike[0].u * 0.7f;
    srand(5);
    float s = 80;
    for (int k = 0; k < NCARS; k++) {
        s += 35 + rand() % 25;
        car[k].s = s;
        car[k].u = ((rand() % 3) - 1) *
4.0f;           // lane -4, 0 or 4
        bool truck = (rand() % 6 == 0);
        car[k].len = truck ? 8.0f : 4.4f;
        car[k].wid = truck ? 2.5f : 2.0f;
        car[k].hgt = truck ? 3.0f : 1.5f;
        car[k].speed = truck ? 8 + rand() % 3 : 10 +
rand() % 6;
        car[k].r = 0.2f + (rand() % 80) / 100.0f;
        car[k].g = 0.2f + (rand() % 80) / 100.0f;
        car[k].b = 0.2f + (rand() % 80) / 100.0f;
    }
    srand(1234);
}

//
------------------------------------------------------
-------- fighting
void hurt(Bike &b, float dmg) {
    if (b.invul > 0 || b.stun > 0) return;
    b.health -= dmg;
    b.invul = 0.5f;
    if (b.health <= 0) { b.stun = 2.5f; b.health = 60;
b.speed = 0; }   // crashed, then respawn
}

void kick(int i, int side) {            // side = -1
left, +1 right
    Bike &b = bike[i];
    if (b.cooldown > 0 || b.stun > 0) return;
    b.cooldown = 0.6f;  b.kickTimer = 0.3f;  b.kickSide
= side;
    for (int j = 0; j < 4; j++) {
        if (j == i) continue;
        Bike &o = bike[j];
        float ds = fabsf(o.s - b.s);
        float du = (o.u - b.u) * side;
        if (ds < 3.2f && du > 0.3f && du < 3.3f) {
            o.wantU = clampf(o.wantU + side * 1.8f, -5,
5);
            o.speed = fmaxf(0, o.speed - 5);
            o.hit = 0.3f;
            hurt(o, 12);
        }
    }
}

//
------------------------------------------------------
-------- AI rider
void ai(int i, float dt) {
    Bike &b = bike[i];
    b.target = 22 + i + clampf((bike[0].s - b.s) *
0.03f, -2, 4);   // slow rider catches up a bit

    bool danger = false;
// look for traffic ahead
    for (int k = 0; k < NCARS; k++) {
        float ahead = car[k].s - b.s;
        if (ahead > 0 && ahead < 20 && fabsf(car[k].u -
b.u) < 2.6f) {
            danger = true;
            float dir = (b.u < car[k].u) ? -1.0f : 1.0f;
            if (fabsf(b.u + dir * 3.5f) > 5) dir = -dir;
            b.wantU = clampf(b.u + dir * 3.5f, -5, 5);
            if (b.target > car[k].speed + 2) b.target =
car[k].speed + 2;
        }
    }
    if (!danger && rand() % 150 == 0) {
// sometimes go and fight a rival
        int j = rand() % 4;
        if (j != i && fabsf(bike[j].s - b.s) < 12)
            b.wantU = clampf(bike[j].u + (b.u >
bike[j].u ? 1.9f : -1.9f), -5, 5);
    }
    for (int j = 0; j < 4; j++) {
// kick rivals who are beside us
        if (j == i) continue;
        float ds = fabsf(bike[j].s - b.s), du =
bike[j].u - b.u;
        if (ds < 3.2f && fabsf(du) > 0.3f && fabsf(du) <
3.3f && rand() % 100 < 2)
            kick(i, du > 0 ? 1 : -1);
    }
    if (b.nitro > 80 && rand() % 400 == 0) { b.nitro -=
30; b.boost = 1.5f; }
}

//
------------------------------------------------------
-------- update
void moveBike(int i, float dt) {
    Bike &b = bike[i];
    if (b.kickTimer > 0) b.kickTimer -= dt;
    if (b.cooldown > 0)  b.cooldown -= dt;
    if (b.hit > 0)       b.hit -= dt;
    if (b.invul > 0)     b.invul -= dt;
    b.prevU = b.u;

    if (b.stun > 0) { b.stun -= dt; b.speed = 0; b.lean
= 0; return; }
    if (i > 0 && !b.done) ai(i, dt);
    if (b.done) b.target = 8;

    // sideways movement (player is faster and snappy)
    float lim = (i == 0) ? 8.5f : 5.0f;
    b.wantU = clampf(b.wantU, -lim, lim);
    float step = ((i == 0) ? 16 : 9) * dt;
    if (b.u < b.wantU) b.u = fminf(b.u + step, b.wantU);
    else               b.u = fmaxf(b.u - step, b.wantU);

    // forward movement
```

```
    float t = b.target;
    if (b.boost > 0) { t += 12; b.boost -= dt; }
    if (i == 0 && fabsf(b.u) > 6) t = fminf(t, 12);
// off road = slow
    if (b.speed < t) b.speed += 14 * dt; else b.speed -=
18 * dt;
    if (b.speed < 0) b.speed = 0;
    b.s += b.speed * dt;
// player follows the road like everyone else
    b.u = clampf(b.u, -lim, lim);
    b.nitro = fminf(100, b.nitro + 4 * dt);

    float vel = (b.u - b.prevU) / fmaxf(dt, 0.001f);
// leaning
    b.lean += (clampf(vel * 1.5f, -22, 22) - b.lean) *
(1.0f - expf(-12.0f * dt));
}

void update(float dt) {
    clockT += dt;
    if (state == 2) { endT += dt; if (endT > 8.0f)
{ resetGame(); return; } }
    if (state == 0) {
        if (clockT > 3.5f) { state = 1; bike[0].target =
24; }
        return;
    }
    if (state == 1) raceTime += dt;

    Bike &pl = bike[0];
// player controls (keys are held down)
    if (pl.stun <= 0 && !pl.done) {
        int steer = (keys['d'] ? 1 : 0) - (keys['a'] ? 1
: 0);
        if (steer)          pl.wantU = clampf(pl.u +
steer * 4.0f, -8.5f, 8.5f);   // go immediately
        else if (lastSteer) pl.wantU = pl.u;
// stop when key released
        lastSteer = steer;
        if (keys['w']) pl.target = fminf(pl.target + 16
* dt, 32);
        if (keys['s']) pl.target = fmaxf(pl.target - 16
* dt, 0);
    }

    for (int i = 0; i < 4; i++) moveBike(i, dt);
    for (int k = 0; k < NCARS; k++) car[k].s +=
car[k].speed * dt;

    // bike <-> traffic
    for (int i = 0; i < 4; i++) {
        Bike &b = bike[i];
        for (int k = 0; k < NCARS; k++) {
            float rel = car[k].s - b.s, half =
car[k].len / 2 + 1;
            if (fabsf(rel) < half && fabsf(car[k].u -
b.u) < car[k].wid / 2 + 0.6f && b.stun <= 0) {
                if (rel > 0) {
// we hit the back of a car
                    b.s = car[k].s - half;
                    b.speed = fminf(b.speed,
car[k].speed * 0.5f);
                    hurt(b, 15);
                } else {
// we are beside the car
                    b.u += (b.u > car[k].u ? 1 : -1) * 6
* dt;
                }
            }
        }
    }
    // bike <-> bike
    for (int i = 0; i < 4; i++) for (int j = i + 1; j <
4; j++) {
        Bike &a = bike[i], &b = bike[j];
        if (fabsf(a.s - b.s) < 2.2f && fabsf(a.u - b.u)
< 1.0f) {
            float push = (a.u >= b.u) ? 0.08f : -0.08f;
            a.u += push;  b.u -= push;
            a.wantU = clampf(a.wantU + push * 3, -5, 5);
            b.wantU = clampf(b.wantU - push * 3, -5, 5);
        }
    }
    // finish line
    for (int i = 0; i < 4; i++) {
        if (!bike[i].done && bike[i].s >= TRACK) {
            bike[i].done = 1;
            bike[i].place = ++finishedCount;
            if (i == 0) {
                state = 2;
                printf("You finished number %d in %.1f
seconds\n", bike[i].place, raceTime);
            }
        }
    }
}

//
------------------------------------------------------
-------- ranking
bool isAhead(int j, int i) {                 // is bike
j ahead of bike i ?
    if (bike[j].done && bike[i].done) return
bike[j].place < bike[i].place;
    if (bike[j].done) return true;
    if (bike[i].done) return false;
    return bike[j].s > bike[i].s;
}
int rankOf(int i) {
    int r = 1;
    for (int j = 0; j < 4; j++) if (j != i && isAhead(j,
i)) r++;
    return r;
}

//
======================================================
========= DRAWING
// A box that stands on y = 0, centred in x and z.
void box(float w, float h, float d, float r, float g,
float b) {
```

```
    float x = w / 2, z = d / 2;
    glBegin(GL_QUADS);
    glColor3f(r, g, b);
// top
    glVertex3f(-x, h, z);  glVertex3f(x, h, z);
glVertex3f(x, h, -z);  glVertex3f(-x, h, -z);
    glColor3f(r * 0.75f, g * 0.75f, b * 0.75f);
// back (+z)
    glVertex3f(-x, 0, z);  glVertex3f(x, 0, z);
glVertex3f(x, h, z);  glVertex3f(-x, h, z);
    glColor3f(r * 0.6f, g * 0.6f, b * 0.6f);
// front (-z)
    glVertex3f(-x, 0, -z); glVertex3f(x, 0, -z);
glVertex3f(x, h, -z); glVertex3f(-x, h, -z);
    glColor3f(r * 0.85f, g * 0.85f, b * 0.85f);
// right side (+x)
    glVertex3f(x, 0, z);   glVertex3f(x, 0, -z);
glVertex3f(x, h, -z); glVertex3f(x, h, z);
    glColor3f(r * 0.65f, g * 0.65f, b * 0.65f);
// left side (-x)
    glVertex3f(-x, 0, z);  glVertex3f(-x, 0, -z);
glVertex3f(-x, h, -z); glVertex3f(-x, h, z);
    glEnd();
}
// A box placed at (x,y,z)
void part(float x, float y, float z, float w, float h,
float d, float r, float g, float b) {
    glPushMatrix();
    glTranslatef(x, y, z);
    box(w, h, d, r, g, b);
    glPopMatrix();
}
// A round shape (polygon) facing the camera
void disc(float x, float y, float z, float rad, float r,
float g, float b) {
    glColor3f(r, g, b);
    glBegin(GL_POLYGON);
    for (int a = 0; a < 360; a += 20)
        glVertex3f(x + rad * cosf(a * PI / 180), y + rad
* sinf(a * PI / 180), z);
    glEnd();
}
// A flat strip of road: lateral range a..b from road
centre, from s to s+len
void strip(float a, float b, float s, float len, float
y) {
    float x1 = roadX(s), x2 = roadX(s + len);
    glBegin(GL_QUADS);
    glVertex3f(x1 + a, y, -s);        glVertex3f(x1 + b,
y, -s);
    glVertex3f(x2 + b, y, -s - len);  glVertex3f(x2 + a,
y, -s - len);
    glEnd();
}

//
------------------------------------------------------
-------- road, buildings, trees
void drawRoad(float s0, float s1) {
    s0 = floorf(s0 / 4) * 4;
    glColor3f(0.25f, 0.62f, 0.25f);
// grass
    for (float s = s0; s < s1; s += 4) strip(-80, 80, s,
4, -0.02f);
    glColor3f(0.68f, 0.68f, 0.64f);
// sidewalks
    for (float s = s0; s < s1; s += 4) { strip(-9, -6,
s, 4, 0); strip(6, 9, s, 4, 0); }
    glColor3f(0.2f, 0.2f, 0.22f);
// asphalt
    for (float s = s0; s < s1; s += 4) strip(-6, 6, s,
4, 0);
    glColor3f(1, 1, 1);
// white lines
    for (float s = s0; s < s1; s += 4) {
        strip(-5.7f, -5.5f, s, 4, 0.01f);  strip(5.5f,
5.7f, s, 4, 0.01f);
        if (((int)(s / 4)) % 2 == 0) { strip(-2.08f, -
1.92f, s, 4, 0.01f); strip(1.92f, 2.08f, s, 4, 0.01f); }
    }
}

void drawLine(float s) {                              //
checkered start / finish line (flat)
    for (int i = 0; i < 12; i++) for (int j = 0; j < 2;
j++) {
        if ((i + j) % 2) glColor3f(0, 0, 0); else
glColor3f(1, 1, 1);
        strip(-6 + i, -5 + i, s + j, 1, 0.02f);
    }
}
void drawArch(float s) {                              //
finish arch
    part(roadX(s) - 6.8f, 0, -s, 0.5f, 7, 0.5f, 0.8f,
0.8f, 0.8f);
    part(roadX(s) + 6.8f, 0, -s, 0.5f, 7, 0.5f, 0.8f,
0.8f, 0.8f);
    for (int i = 0; i < 14; i++)
        part(roadX(s) - 6.5f + i, 7, -s, 1, 1, 0.4f, (i
% 2) ? 0 : 1, (i % 2) ? 0 : 1, (i % 2) ? 0 : 1);
}

unsigned hashi(int i, int side) {
    unsigned x = i * 374761393u + (side + 2) *
668265263u;
    x = (x ^ (x >> 13)) * 1274126177u;
    return x ^ (x >> 16);
}

void drawBuilding(int i, int side) {
    float s = i * 14.0f;
    unsigned h = hashi(i, side);
    float w = 7 + h % 6, height = 5 + (h >> 5) % 9,
depth = 9;
    float wall[6][3]  = { {0.8f,0.55f,0.45f},
{0.85f,0.8f,0.65f}, {0.55f,0.62f,0.75f},
                          {0.7f,0.7f,0.72f},
{0.75f,0.45f,0.4f}, {0.55f,0.7f,0.55f} };
    float shop[5][3]  = { {0.9f,0.15f,0.15f},
{0.15f,0.6f,0.9f}, {0.95f,0.7f,0.1f},
```

```
                        {0.2f,0.75f,0.3f},
{0.85f,0.3f,0.7f} };
    int c = (h >> 9) % 6, a = (h >> 13) % 5;

    glPushMatrix();
    glTranslatef(roadX(s) + side * (10.5f + w / 2), 0, -
s);
    box(w, height, depth, wall[c][0], wall[c][1],
wall[c][2]);

    float fx = -side * (w / 2 + 0.03f);
// the wall that faces the road
    glBegin(GL_QUADS);
    glColor3f(0.3f, 0.5f, 0.65f);
// shop glass
    glVertex3f(fx, 0.2f, -3.8f);  glVertex3f(fx, 0.2f,
1.2f);  glVertex3f(fx, 2.4f, 1.2f);  glVertex3f(fx,
2.4f, -3.8f);
    glColor3f(0.35f, 0.2f, 0.1f);
// door
    glVertex3f(fx, 0, 2.0f);  glVertex3f(fx, 0, 3.6f);
glVertex3f(fx, 2.4f, 3.6f);  glVertex3f(fx, 2.4f, 2.0f);
    glColor3f(0.65f, 0.82f, 0.95f);
// upper windows
    for (float y = 3.4f; y + 1.3f < height; y += 2.3f)
        for (float z0 = -4.0f; z0 + 1.4f < 4.5f; z0 +=
2.5f) {
            glVertex3f(fx, y, z0);  glVertex3f(fx, y, z0
+ 1.4f);
            glVertex3f(fx, y + 1.3f, z0 + 1.4f);
glVertex3f(fx, y + 1.3f, z0);
        }
    glEnd();
    part(-side * (w / 2 + 0.9f), 2.6f, 0, 1.8f, 0.25f,
depth * 0.95f, shop[a][0], shop[a][1], shop[a][2]);   //
awning
    part(-side * (w / 2 - 0.4f), height, 0, 0.3f, 1.4f,
depth * 0.7f,
         shop[(a + 2) % 5][0], shop[(a + 2) % 5][1],
shop[(a + 2) % 5][2]);
// roof sign
    glPopMatrix();
}

void drawTreeOrLamp(int i, int side) {
    float s = i * 14.0f + 7;
    float x = roadX(s) + side * 7.6f;
    if (hashi(i, side + 7) % 2)
{                       // tree
        part(x, 0, -s, 0.4f, 2.2f, 0.4f, 0.4f, 0.25f,
0.1f);
        part(x, 2.0f, -s, 2.4f, 2.0f, 2.4f, 0.1f, 0.5f,
0.15f);
        part(x, 3.8f, -s, 1.6f, 1.4f, 1.6f, 0.15f, 0.6f,
0.2f);
    } else
{                                           // street
lamp
        part(x, 0, -s, 0.2f, 6, 0.2f, 0.25f, 0.25f,
0.28f);
        part(x - side * 0.8f, 6, -s, 1.6f, 0.15f, 0.2f,
0.25f, 0.25f, 0.28f);
        part(x - side * 1.5f, 5.8f, -s, 0.6f, 0.2f,
0.4f, 1, 0.95f, 0.5f);
    }
}

//
------------------------------------------------------
-------- vehicles
void drawCar(Car &c) {
    glPushMatrix();
    glTranslatef(roadX(c.s) + c.u, 0, -c.s);
    glRotatef(-roadAngle(c.s), 0, 1, 0);
// follow the road bend
    for (int sx = -1; sx <= 1; sx += 2) for (int sz = -
1; sz <= 1; sz += 2)      // wheels
        part(sx * c.wid * 0.5f, 0, sz * c.len * 0.3f,
0.3f, 0.8f, 0.9f, 0.05f, 0.05f, 0.05f);
    if (c.len < 6) {
// car
        part(0, 0.3f, 0, c.wid * 0.96f, 0.7f, c.len,
c.r, c.g, c.b);
        part(0, 1.0f, 0.3f, c.wid * 0.85f, 0.65f, c.len
* 0.5f, 0.2f, 0.25f, 0.35f);
    } else {
// truck
        part(0, 0.4f, 1.25f, c.wid, c.hgt - 0.4f, c.len
- 2.5f, 0.85f, 0.85f, 0.85f);
        part(0, 0.4f, -2.75f, c.wid, c.hgt * 0.65f,
2.5f, c.r, c.g, c.b);
    }
    glBegin(GL_QUADS);
// red tail lights
    glColor3f(1, 0.1f, 0.1f);
    float z = c.len / 2 + 0.02f;
    for (int k = -1; k <= 1; k += 2) {
        glVertex3f(k * c.wid * 0.35f - 0.2f, 0.5f, z);
glVertex3f(k * c.wid * 0.35f + 0.2f, 0.5f, z);
        glVertex3f(k * c.wid * 0.35f + 0.2f, 0.8f, z);
glVertex3f(k * c.wid * 0.35f - 0.2f, 0.8f, z);
    }
    glEnd();
    glPopMatrix();
}

void drawBike(int id) {
    Bike &b = bike[id];
    float jr = b.b * 0.5f + 0.1f, jg = b.r * 0.4f +
0.1f, jb = b.g * 0.5f + 0.1f;   // jacket colour
    glPushMatrix();
    glTranslatef(roadX(b.s) + b.u, 0, -b.s);
    glRotatef(-roadAngle(b.s), 0, 1, 0);
// follow the road bend
    glScalef(1.4f, 1.4f, 1.4f);
// size of the bike

    glBegin(GL_QUADS);
// shadow
    glColor3f(0.1f, 0.15f, 0.1f);
```

```
    glVertex3f(-0.6f, 0.03f, -1.6f);  glVertex3f(0.6f,
0.03f, -1.6f);
    glVertex3f(0.6f, 0.03f, 1.6f);    glVertex3f(-0.6f,
0.03f, 1.6f);
    glEnd();

    if (b.stun > 0) glRotatef(80, 0, 0, 1);
// crashed: lying down
    else            glRotatef(-b.lean, 0, 0, 1);
// lean while turning

    // ---- front part
    part(0, 0, -1.0f, 0.26f, 0.9f, 0.9f, 0.05f, 0.05f,
0.05f);            // front wheel
    part(0, 0.9f, -1.0f, 0.08f, 0.02f, 0.6f, 0.8f, 0.8f,
0.85f);          // front rim
    part(0, 0.5f, -0.85f, 0.5f, 0.45f, 0.5f, b.r, b.g,
b.b);              // front fairing
    part(0, 0.95f, -0.65f, 0.4f, 0.4f, 0.05f, 0.6f,
0.8f, 0.9f);          // wind screen
    part(0, 1.05f, -0.4f, 1.0f, 0.1f, 0.1f, 0.75f,
0.75f, 0.8f);          // handle bar
    part(-0.55f, 1.1f, -0.45f, 0.12f, 0.18f, 0.1f, b.r,
b.g, b.b);        // mirrors
    part( 0.55f, 1.1f, -0.45f, 0.12f, 0.18f, 0.1f, b.r,
b.g, b.b);
    // ---- middle part
    part(0, 0.55f, -0.2f, 0.55f, 0.55f, 0.8f, b.r, b.g,
b.b);             // fuel tank
    part(0, 0.5f, 0.55f, 0.5f, 0.45f, 0.9f, b.r * 0.85f,
b.g * 0.85f, b.b * 0.85f);   // tail
    part(0, 0.95f, 0.55f, 0.12f, 0.02f, 0.9f, 1, 1, 1);
// white stripe
    part(0, 0.97f, 0.45f, 0.4f, 0.12f, 0.6f, 0.08f,
0.08f, 0.08f);        // seat
    part(0, 0.5f, 1.0f, 0.5f, 0.3f, 0.4f, b.r * 0.85f,
b.g * 0.85f, b.b * 0.85f);   // rear mud guard
    // ---- rider legs (kick animation)
    for (int side = -1; side <= 1; side += 2) {
        float ang = 0;
        if (b.kickTimer > 0 && b.kickSide == side) ang =
side * 80 * sinf(b.kickTimer / 0.3f * PI);
        glPushMatrix();
        glTranslatef(side * 0.34f, 1.0f, 0.3f);
        glRotatef(ang, 0, 0, 1);
        glTranslatef(0, -0.7f, 0);
        box(0.2f, 0.7f, 0.28f, 0.12f, 0.12f, 0.35f);
// leg
        part(0, -0.05f, -0.1f, 0.24f, 0.15f, 0.45f,
0.05f, 0.05f, 0.05f);  // boot
        glPopMatrix();
    }
    // ---- rider body
    part(0, 1.0f, 0.35f, 0.6f, 0.85f, 0.38f, jr, jg,
jb);                  // jacket
    part(0, 1.05f, 0.55f, 0.14f, 0.7f, 0.02f, 1, 1, 1);
// back stripe
    part(-0.4f, 1.4f, -0.05f, 0.14f, 0.14f, 0.95f, jr,
jg, jb);            // arms
    part( 0.4f, 1.4f, -0.05f, 0.14f, 0.14f, 0.95f, jr,
jg, jb);
    part(0, 1.78f, 0.3f, 0.5f, 0.5f, 0.55f, b.r, b.g,
b.b);                // helmet
    part(0, 1.95f, 0.0f, 0.4f, 0.18f, 0.1f, 0.05f,
0.05f, 0.1f);           // dark visor
    part(0, 2.28f, 0.3f, 0.1f, 0.02f, 0.55f, 1, 1, 1);
// helmet stripe
    // ---- rear part (nearest to camera, so drawn last)
    part(0, 0, 1.05f, 0.3f, 0.9f, 0.9f, 0.05f, 0.05f,
0.05f);             // rear wheel
    part(0, 0.9f, 1.05f, 0.08f, 0.02f, 0.6f, 0.8f, 0.8f,
0.85f);          // rear rim
    part(-0.38f, 0.2f, 0.9f, 0.14f, 0.22f, 1.1f, 0.55f,
0.55f, 0.6f);     // exhaust pipes
    part( 0.38f, 0.2f, 0.9f, 0.14f, 0.22f, 1.1f, 0.55f,
0.55f, 0.6f);
    part(0, 0.95f, 1.35f, 0.3f, 0.12f, 0.14f, 1, 0.1f,
0.1f);              // red tail light

    if (b.boost > 0) {
// nitro flame
        disc(0.38f, 0.3f, 1.7f, 0.3f, 1, 0.5f, 0.1f);
        disc(-0.38f, 0.3f, 1.7f, 0.3f, 1, 0.5f, 0.1f);
        disc(0.38f, 0.3f, 1.8f, 0.15f, 1, 1, 0.3f);
        disc(-0.38f, 0.3f, 1.8f, 0.15f, 1, 1, 0.3f);
    }
    if (b.hit > 0) {
// hit star
        glColor3f(1, 0.9f, 0.1f);
        glBegin(GL_POLYGON);
        for (int a = 0; a < 360; a += 45) {
            float r = (a % 90 == 0) ? 0.9f : 0.4f;
            glVertex3f(r * cosf(a * PI / 180), 2.4f + r
* sinf(a * PI / 180), 1.2f);
        }
        glEnd();
    }
    glPopMatrix();
}

//
------------------------------------------------------
-------- whole world
void drawWorld() {
    float ps = bike[0].s;

glDisable(GL_DEPTH_TEST);                          //
flat ground first, everything else is drawn on top
    drawRoad(ps - 20, ps + 260);
    drawLine(0);
    drawLine(TRACK);

glEnable(GL_DEPTH_TEST);                           //
now real 3D objects

    drawArch(TRACK);
    int i0 = (int)floorf((ps - 25) / 14), i1 = (int)((ps
+ 200) / 14);
    for (int i = i0; i <= i1; i++) {
        if (i < -2) continue;
        for (int side = -1; side <= 1; side += 2) {
            drawBuilding(i, side);
```

```
            drawTreeOrLamp(i, side);
        }
    }
    for (int k = 0; k < NCARS; k++)
        if (car[k].s > ps - 20 && car[k].s < ps + 200)
drawCar(car[k]);
    for (int i = 0; i < 4; i++) drawBike(i);
}

//
------------------------------------------------------------
-------- HUD (2D)
void rectQ(float x, float y, float w, float h, float r,
float g, float b) {
    glColor3f(r, g, b);
    glBegin(GL_QUADS);
    glVertex3f(x, y, 0);  glVertex3f(x + w, y, 0);
glVertex3f(x + w, y + h, 0);  glVertex3f(x, y + h, 0);
    glEnd();
}

void drawHUD() {
    glDisable(GL_DEPTH_TEST);
    glMatrixMode(GL_PROJECTION);  glLoadIdentity();
glOrtho(0, W, 0, H, -1, 1);
    glMatrixMode(GL_MODELVIEW);   glLoadIdentity();
    Bike &p = bike[0];

    // speed / nitro / health bars (bottom right)
    rectQ(W - 260, 15, 245, 100, 0.1f, 0.1f, 0.1f);
    rectQ(W - 245, 90, 215, 14, 0.3f, 0.3f, 0.3f);
rectQ(W - 245, 90, 215 * clampf(p.speed / 45, 0, 1), 14,
0.2f, 0.9f, 0.2f);
    rectQ(W - 245, 60, 215, 14, 0.3f, 0.3f, 0.3f);
rectQ(W - 245, 60, 215 * p.nitro / 100, 14, 0.2f, 0.6f,
1);
    rectQ(W - 245, 30, 215, 14, 0.3f, 0.3f, 0.3f);
rectQ(W - 245, 30, 215 * p.health / 100, 14, 1 -
p.health / 100, p.health / 100, 0.1f);

    // ranking board (top left): coloured square +
health bar
    rectQ(15, H - 150, 170, 135, 0.1f, 0.1f, 0.1f);
    for (int r = 1; r <= 4; r++)
        for (int i = 0; i < 4; i++)
            if (rankOf(i) == r) {
                float y = H - 20 - r * 30;
                rectQ(25, y, 20, 20, bike[i].r,
bike[i].g, bike[i].b);
                rectQ(55, y + 4, 120, 12, 0.3f, 0.3f,
0.3f);
                rectQ(55, y + 4, 120 * bike[i].health /
100, 12, 0.9f, 0.9f, 0.9f);
            }

    // race progress bar (top centre)
    rectQ(W / 2 - 260, H - 40, 520, 25, 0.1f, 0.1f,
0.1f);
    rectQ(W / 2 - 250, H - 30, 500, 5, 0.8f, 0.8f,
0.8f);
    for (int i = 3; i >= 0; i--) {
        float t = clampf(bike[i].s / TRACK, 0, 1);
        rectQ(W / 2 - 250 + t * 488, H - 36, 12, 17,
bike[i].r, bike[i].g, bike[i].b);
    }

    if (state == 0) {
// start lights
        rectQ(W / 2 - 100, H / 2 - 40, 200, 80, 0.05f,
0.05f, 0.05f);
        int lit = (int)clockT + 1;  if (lit > 3) lit =
3;
        for (int k = 0; k < 3; k++) {
            if (clockT >= 3.0f)  disc(W / 2 - 60 + k *
60, H / 2, 0, 24, 0.1f, 1, 0.1f);       // GO = green
            else if (k < lit)    disc(W / 2 - 60 + k *
60, H / 2, 0, 24, 1, 0.1f, 0.1f);
            else                 disc(W / 2 - 60 + k *
60, H / 2, 0, 24, 0.3f, 0.1f, 0.1f);
        }
    }
    if (state == 2) {
// result medal
        int pl = p.place;
        rectQ(W / 2 - 150, H / 2 - 100, 300, 200, 0.1f,
0.1f, 0.1f);
        if (pl == 1) disc(W / 2, H / 2 + 20, 0, 55, 1,
0.85f, 0.1f);
        else if (pl == 2) disc(W / 2, H / 2 + 20, 0, 55,
0.8f, 0.8f, 0.85f);
        else if (pl == 3) disc(W / 2, H / 2 + 20, 0, 55,
0.8f, 0.5f, 0.2f);
        else disc(W / 2, H / 2 + 20, 0, 55, 0.4f, 0.4f,
0.4f);
        for (int k = 0; k < pl; k++) rectQ(W / 2 - pl *
12 + k * 24, H / 2 - 80, 16, 16, 1, 1, 1);   // place =
number of squares
        glColor3f(1, 1, 1);
        glRasterPos2f(W / 2 - 80, H / 2 - 58);
        const char *msg = "Press R to restart";
        for (const char *c = msg; *c; c++)
glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, *c);
    }
}

//
------------------------------------------------------------
-------- display
void drawSky() {
    glDisable(GL_DEPTH_TEST);
    glMatrixMode(GL_PROJECTION);  glLoadIdentity();
glOrtho(0, W, 0, H, -1, 1);
    glMatrixMode(GL_MODELVIEW);   glLoadIdentity();
    glBegin(GL_QUADS);
    glColor3f(0.85f, 0.93f, 1.0f);  glVertex3f(0, H *
0.45f, 0);  glVertex3f(W, H * 0.45f, 0);
    glColor3f(0.25f, 0.5f, 0.95f);  glVertex3f(W, H, 0);
glVertex3f(0, H, 0);
    glEnd();
    disc(W * 0.8f, H * 0.85f, 0, 40, 1, 0.95f, 0.6f);
// sun
}
```

```
void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    drawSky();

    // Highway-Rider style chase camera: behind, above,
bike in lower part of screen
    Bike &p = bike[0];
    camU += (p.u * 0.7f - camU) * (1.0f - expf(-8.0f *
frameDt));   // smooth lag so bike moves on screen
    float th   = atanf((roadX(p.s + 6) - roadX(p.s - 6))
/ 12.0f);  // road direction
    float camX = roadX(p.s) + camU - CAM_DIST *
sinf(th);

    glMatrixMode(GL_PROJECTION);  glLoadIdentity();
    glFrustum(-0.888, 0.888, -0.577, 0.577, 1.0, 350.0);
    glMatrixMode(GL_MODELVIEW);   glLoadIdentity();
    glRotatef(CAM_PITCH, 1, 0, 0);
// look slightly down
    glRotatef(th * 180.0f / PI, 0, 1, 0);
// turn with the road
    glTranslatef(-camX, -CAM_HEIGHT, p.s - CAM_DIST *
cosf(th));

    drawWorld();
    drawHUD();
    glutSwapBuffers();
}

//
------------------------------------------------------------
-------- keyboard
void keyboard(unsigned char key, int x, int y) {
    if (key >= 'A' && key <= 'Z') key += 32;
    keys[key] = true;
// W A S D are read in update()
    if (key == 27) exit(0);
    if (key == 'r') { resetGame(); return; }
    if (state != 1) return;
    Bike &p = bike[0];
    if (key == ' ' && p.nitro >= 30 && p.stun <= 0)
{ p.nitro -= 30; p.boost = 1.5f; }
    if (key == 'z') kick(0, -1);
    if (key == 'x') kick(0, 1);
}
void keyboardUp(unsigned char key, int x, int y) {
    if (key >= 'A' && key <= 'Z') key += 32;
    keys[key] = false;
}

//
------------------------------------------------------------
-------- timer (animation)
void timer(int v) {
    static int lastMs = 0;
    int now = glutGet(GLUT_ELAPSED_TIME);
    float dt = (lastMs == 0) ? 0.016f : (now - lastMs) /
1000.0f;
    lastMs = now;
    if (dt > 0.05f) dt = 0.05f;
    if (dt < 0.001f) dt = 0.001f;
    frameDt = dt;

    update(dt);
    frameNo++;
    if (frameNo % 10 == 0) {
// info in the title bar
        char t[160];
        sprintf(t, "Moto Racer 3D | Place %d/4 | Speed
%d km/h | Health %d | Nitro %d | Time %.1f",
                rankOf(0), (int)(bike[0].speed * 4),
(int)bike[0].health, (int)bike[0].nitro, raceTime);
        glutSetWindowTitle(t);
    }
    glutPostRedisplay();
    glutTimerFunc(8, timer, 0);
}

int main(int argc, char **argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB |
GLUT_DEPTH);
    glutInitWindowSize(W, H);
    glutInitWindowPosition(100, 50);
    glutCreateWindow("Moto Racer 3D");
    glClearColor(0.85f, 0.93f, 1.0f, 1.0f);
// sky colour
    resetGame();
    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);
    glutKeyboardUpFunc(keyboardUp);
    glutIgnoreKeyRepeat(1);
    glutTimerFunc(8, timer, 0);
    glutMainLoop();
    return 0;
}
```

Output:



Figure-01: Race Start with Countdown Lights



Figure-02: Race Finish with Result Medal

1.4 Instructions:

i. Riding Controls (hold the keys)

- **W:** Increase the wanted speed (accelerate).

- **S:** Decrease the wanted speed (brake).
- **A:** Move the bike to the left lane side.
- **D:** Move the bike to the right lane side.

ii. Boost and Combat

- **Spacebar:** Use nitro (costs 30 nitro; at least 30 must be available). The speed increases by 12 units/s for 1.5 seconds.
- **Z:** Kick the rival on the left side.
- **X:** Kick the rival on the right side.

iii. Game Control

- **R:** Restart the race at any time.
- **ESC:** Exit the game.

iv. Race Rules

- Wait for the three start lights to turn green; the bikes cannot move during the countdown.
- Avoid cars and trucks: hitting the back of a vehicle reduces health and speed.
- Staying on the asphalt is faster; riding on the sidewalk or grass slows the player.
- The first rider to cross the finish line at 2200 units wins. The result box shows the final position as a medal.

v. Compile Instructions

- Linux: `g++ moto_racer_simple.cpp -o moto -lGL -lGLU -lglut -lm`
- Windows (MinGW): `g++ moto_racer_simple.cpp -o moto.exe -lfreeglut -lopengl32 -lglu32`

1.5 Discussion:

Moto Racer 3D successfully demonstrates a complete 3D racing game developed with native C++ and OpenGL/GLUT without any external assets. The game includes a procedurally curved road, a city generated from simple boxes, a smooth chase camera, responsive controls, traffic, three AI riders, a health and combat system, and a clear HUD. However, the project has some limitations. It uses the legacy fixed-function pipeline without real lighting, shadows, or textures; all objects are low-polygon boxes; buildings are not rotated with the road bends; collision detection is approximate and two-dimensional; the AI uses simple rules; and the game has no sound effects or music.

In the future, the project can be improved by using modern OpenGL with GLSL shaders for lighting, shadow mapping, and textures; adding engine sound and music with an audio library; adding more tracks with a track editor and different weather or time of day; improving bike

physics with banking and suspension; adding better AI with path planning; saving best lap times; and adding a multiplayer mode. Overall, the project validates important computer graphics concepts such as modelling, transformations, projection, depth testing, animation, and real-time interaction, and provides a strong foundation for developing larger racing games.